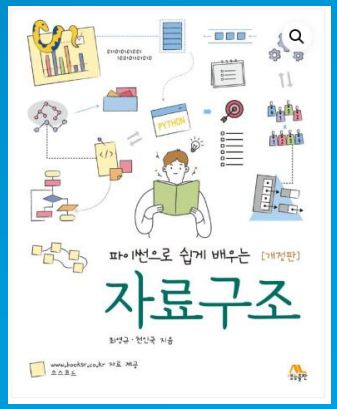
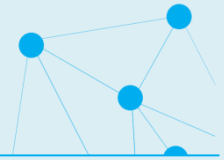


CHAPTER 06

연결된 구조

6장. 연결된 구조



6.1 연결된 구조란?

6.2 단순 연결 구조의 응용: 연결된 스택

6.3 단순 연결 구조의 응용: 연결된 리스트

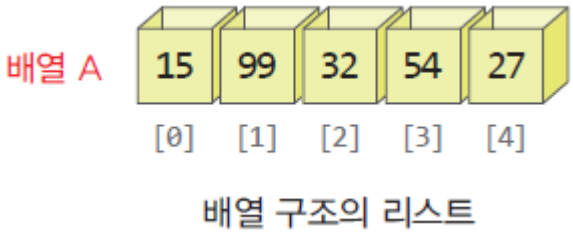
6.4 원형 연결 구조의 응용: 연결된 큐

6.5 이중 연결 구조의 응용: 연결된 덱

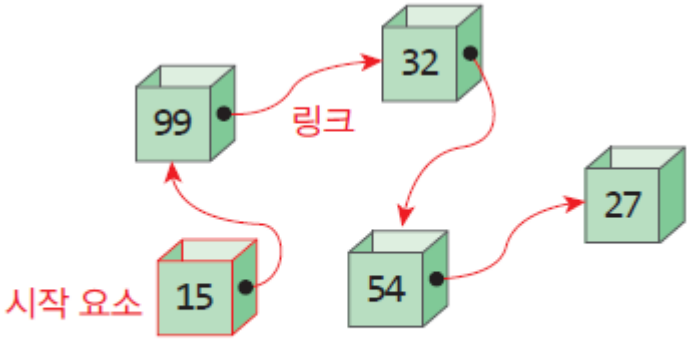
6.1 연결된 구조란?

- 배열 구조
  - 모든 데이터를 연속된 메모리 공간에 저장
  - n번째 자료의 위치를 금방 찾을 수 있음: O(1)

- 연결된 구조 (linked structure)
  - 데이터를 한군데 모아두는 것을 포기
  - 흩어진 데이터를 링크로 연결해서 관리
  - n번째 자료의 위치를 찾으려면? O(n)



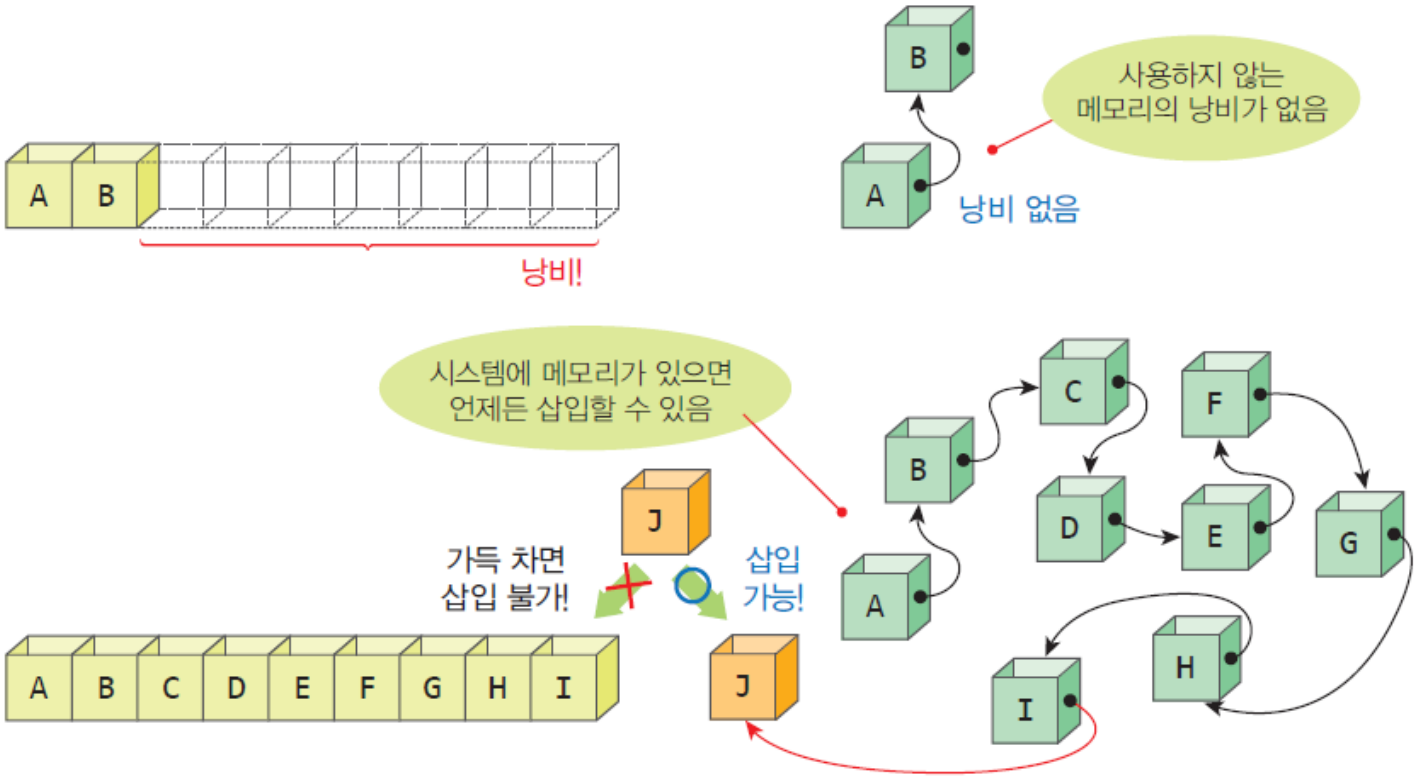
배열 구조의 리스트



연결된 구조의 리스트

연결된 구조의 특징

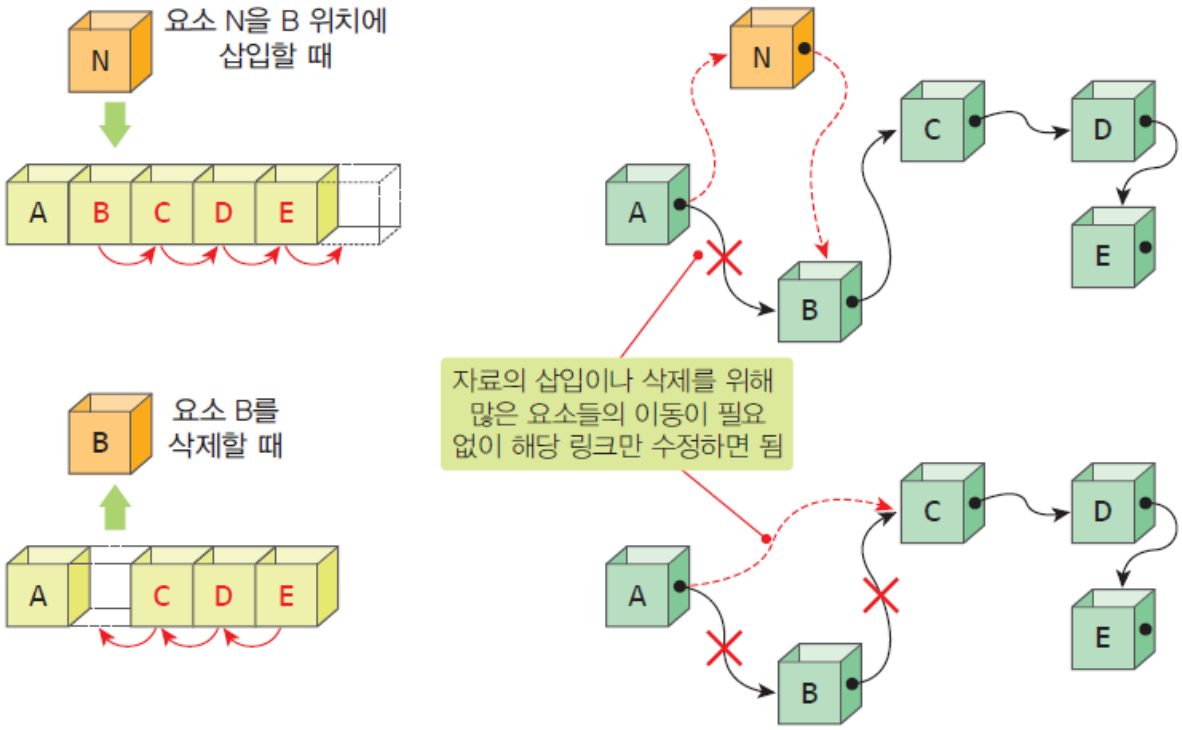
- 메모리 낭비가 없고 용량이 고정되지 않음.



(a) 배열 구조에서는 메모리의 낭비나 용량 고정 문제가 발생한다.

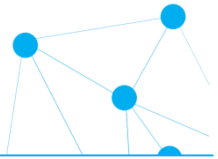
(b) 연결된 구조에서는 메모리 낭비가 없고, 시스템에 메모리가 있으면 언제든 삽입이 가능하다.

- 중간에 자료를 삽입하거나 삭제하는 것이 용이

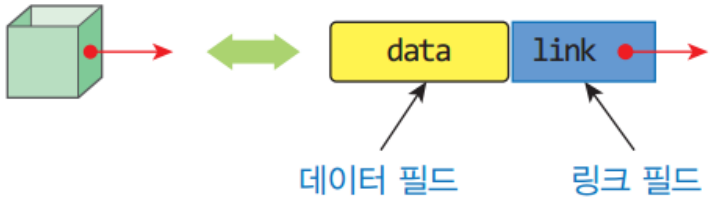


- n번째 항목 접근에 $O(n)$의 시간이 걸림.

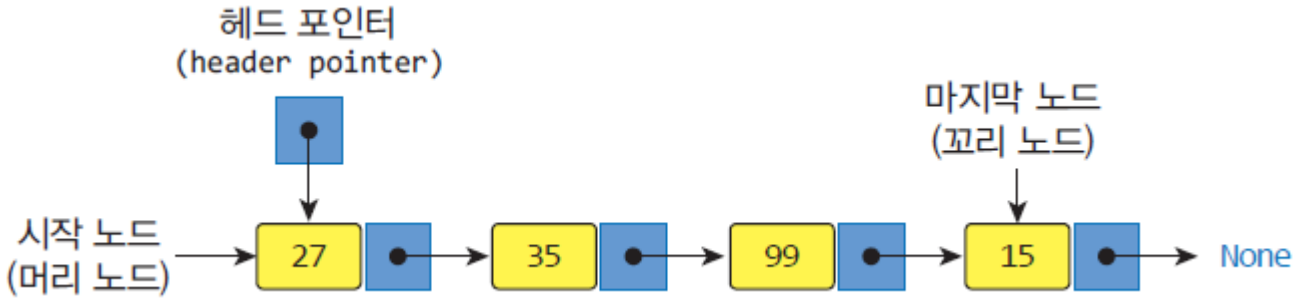
연결된 구조의 용어



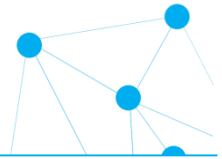
- 노드 (node)
  - 데이터 필드(data field)
  - 하나 이상의 링크 필드(link field)



- 시작/꼬리 노드, 헤드 포인터 (head pointer)



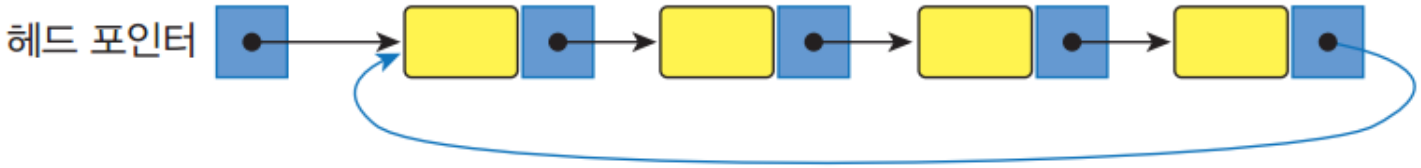
연결 리스트의 종류



- 단순 연결 리스트(singly linked list)



- 원형 연결 리스트(circular linked list)

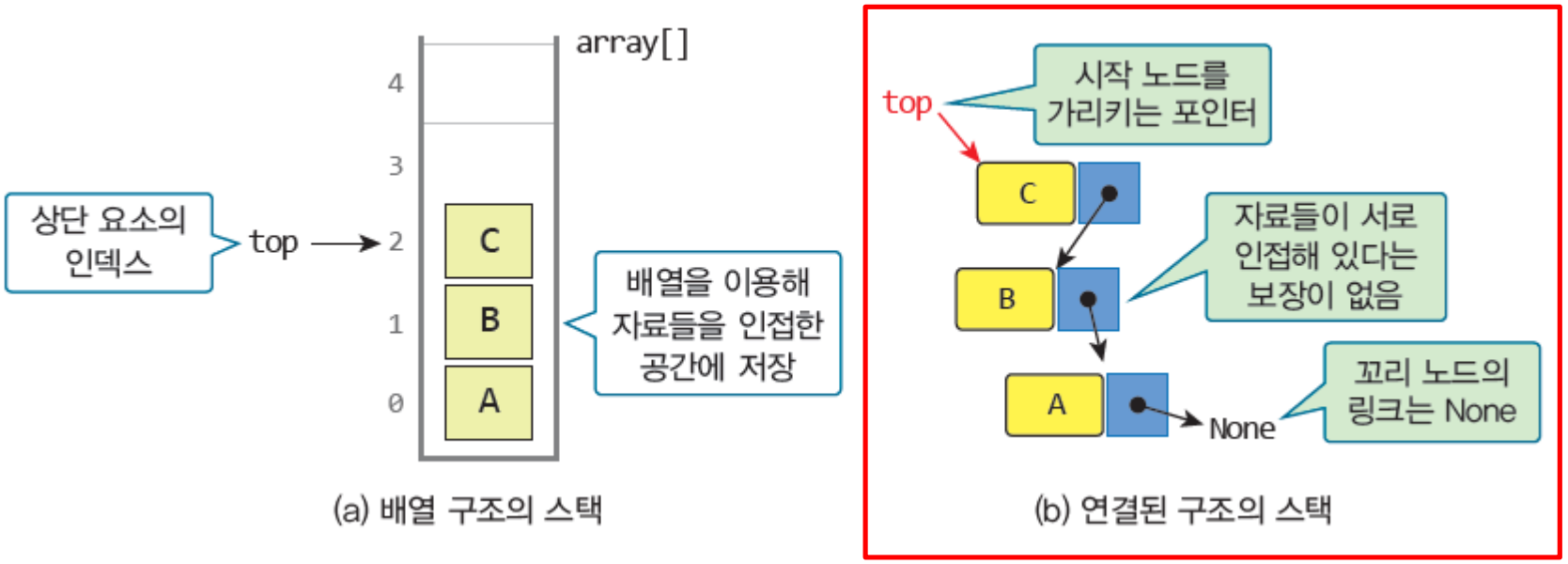


- 이중 연결 리스트(doubly linked list)

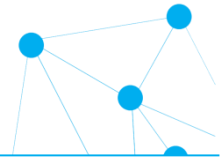


6.2 단순연결리스트 응용: 연결된 스택

- 배열 구조의 스택(4장)과 연결된 스택의 구조 비교



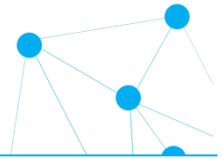
Node 클래스



- 단순연결구조의 노드 클래스

```
class Node:                                  # 단순연결리스트를 위한 노드 클래스
    def __init__ (self, elem, link=None):    # 생성자. 디폴트 인수 사용
        self.data = elem                     # 데이터 멤버 생성 및 초기화
        self.link = link                     # 링크 생성 및 초기화
```

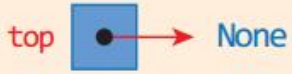
LinkedStack 클래스



- 클래스 정의와 생성자

```
class LinkedStack :
    def __init__( self ):  # 생성자
        self.top = None    # top 생성 및 초기화

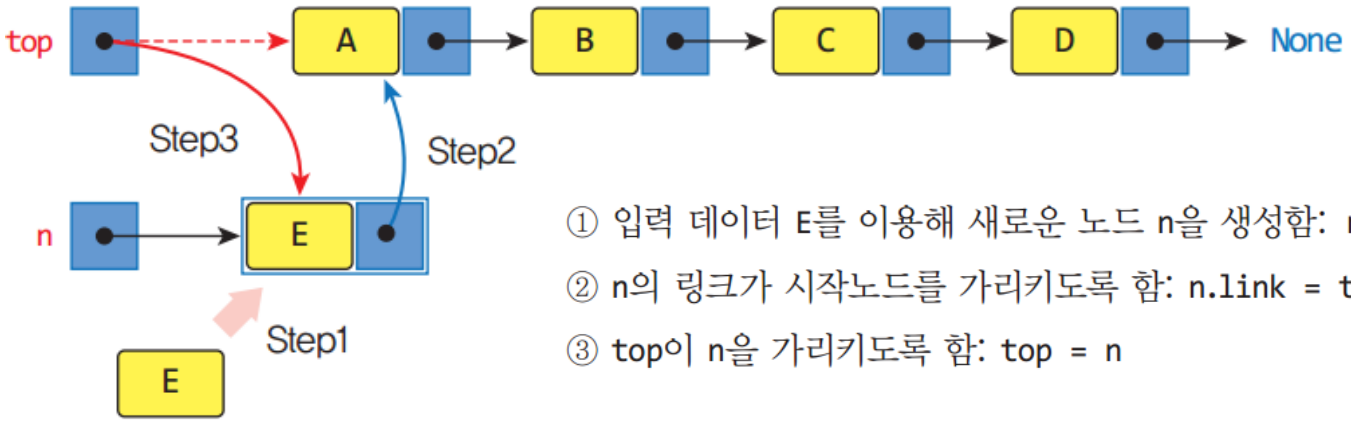
    def isEmpty( self ): return self.top == None      # 공백상태 검사
    def clear( self ): self.top = None                # 스택 초기화
```



  - 연결된 구조에서 포화 상태는 의미 없음 → 항상 False

삽입 연산

- 맨 앞에 새로운 노드를 추가



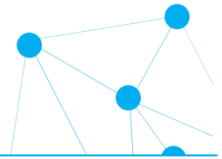
① 입력 데이터 E를 이용해 새로운 노드 n을 생성함: n = Node(E)

② n의 링크가 시작노드를 가리키도록 함: n.link = top

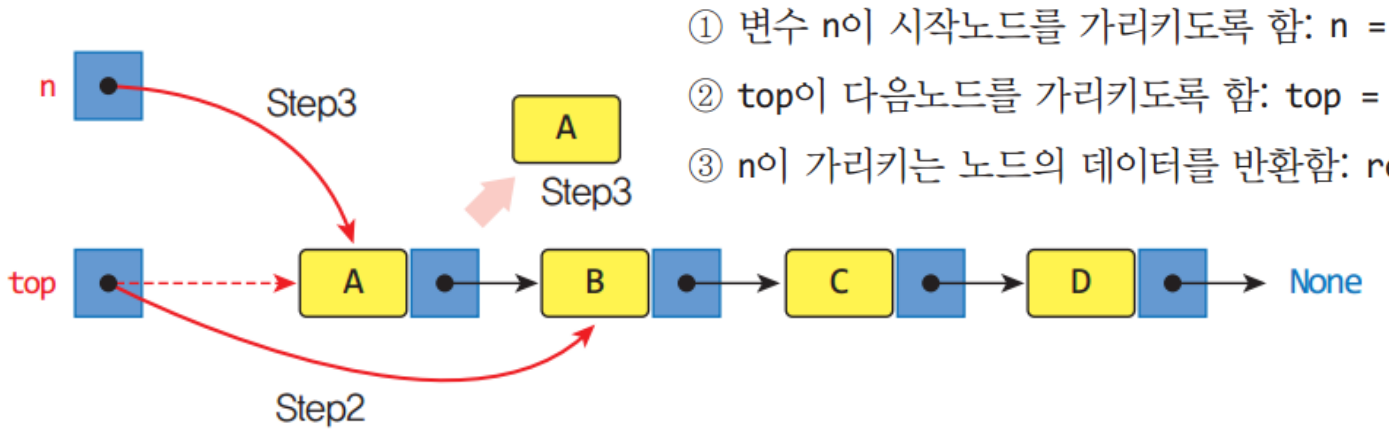
③ top이 n을 가리키도록 함: top = n

```
def push( self, item ):                # 연결된 스택의 삽입연산
    n = Node(item, self.top)           # Step1 + Step2
    self.top = n                       # Step3
```

삭제 연산



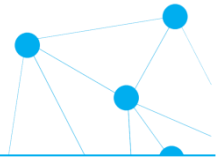
- 시작 노드의 데이터를 반환



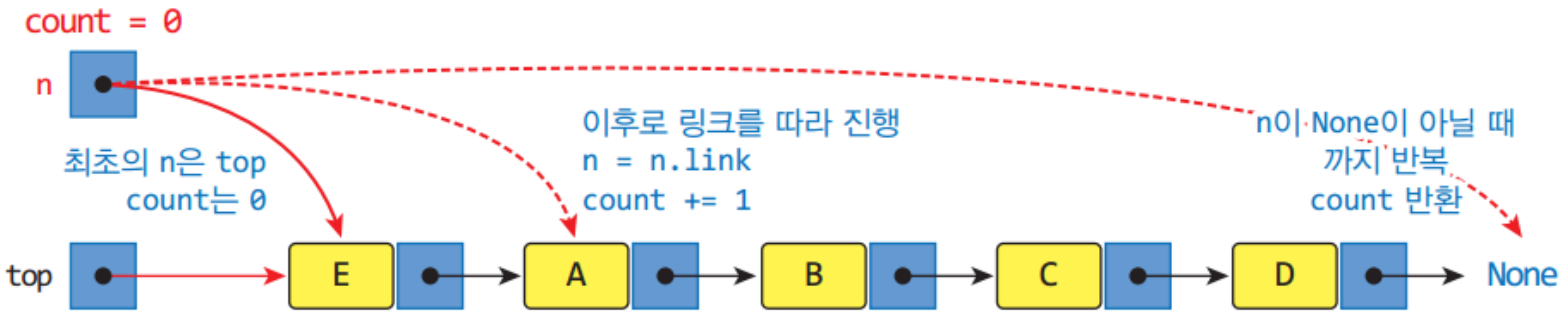

```
def pop( self ):                    # 연결된 스택의 삭제연산
    if not self.isEmpty():          # 공백이 아니면
        n = self.top                # Step1
        self.top = self.n.link      # Step2
        return n.data               # Step3
```

- 파이썬에서는 메모리 해제를 신경 쓸 필요 없음!

기타 연산들



- peek() : 시작 노드의 데이터를 반환
- 요소의 수를 구하는 size()
  - 코드 6.3

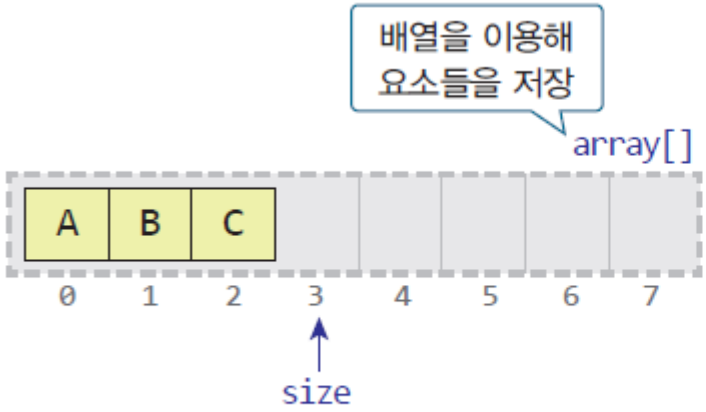


  - 노드의 수를 저장하는 count를 LinkedStack의 멤버 변수로 추가한다면?

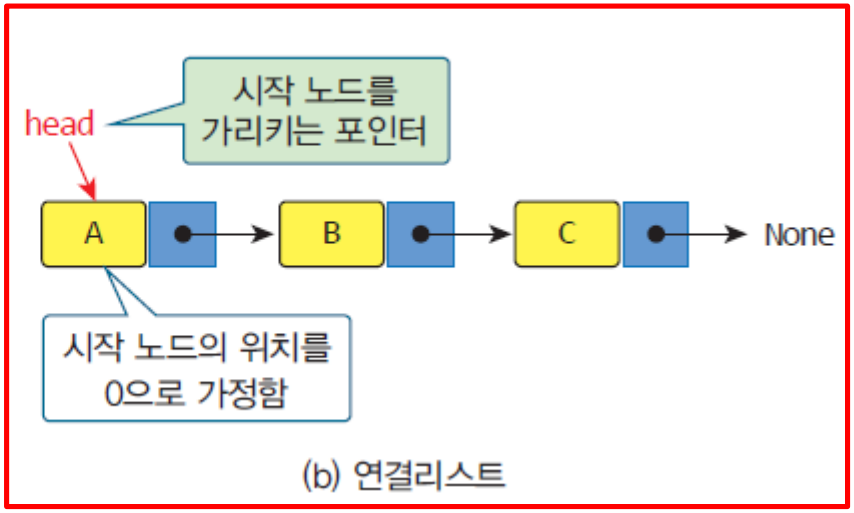
- 문자열 변환을 위한 str연산자 중복
  - 코드 6.4

6.3 단순연결리스트 응용: 연결 리스트

- 배열을 이용한 리스트(3장)와 연결리스트의 구조 비교

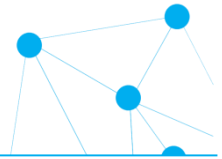


(a) 배열 구조의 리스트



(b) 연결리스트

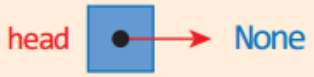
LinkedList 클래스



- 클래스 정의와 생성자

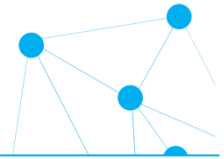
```
class LinkedList:                                   # 연결된 리스트 클래스
    def __init__( self ):
        self.head = None

    def isEmpty( self ): return self.head == None   # 공백상태 검사
    def clear( self ) : self.head = None            # 리스트 초기화
    def size( self ) : ...                          # self.top->self.head로 수정. 코드 동일
    def display(self, msg):...                      # self.top->self.head로 수정. 코드 동일
```



  - isEmpty(), isFull(), size() 등은 연결된 스택에서와 유사함
    - 시작 노드를 가리키는 top을 head로 바꾸기만 하면 됨

항목 참조, 노드 참조

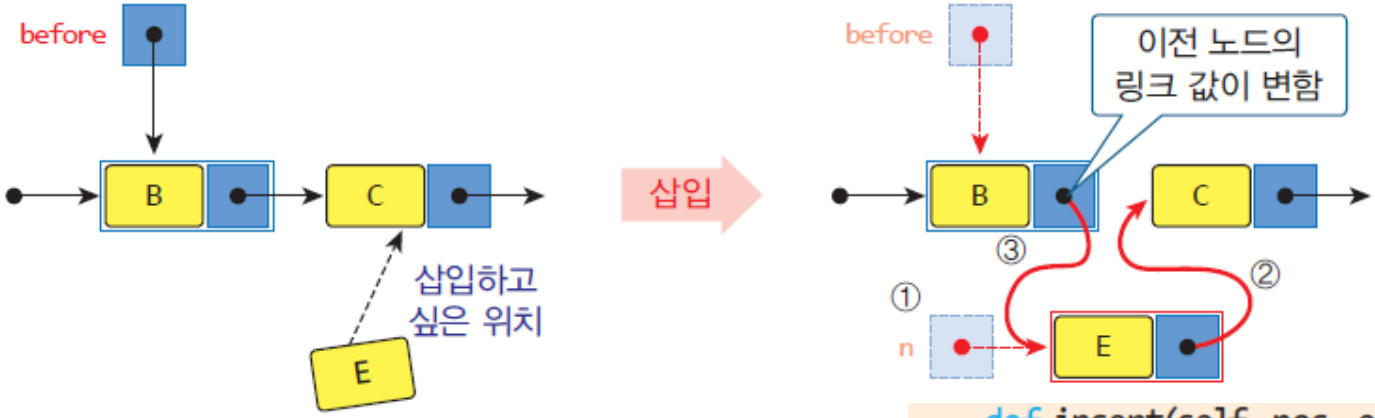


- pos번째 항목 참조를 위해서는 먼저 노드를 찾아야 함
  - getNode(pos) 메소드 추가
  - getEntry(pos)에서는 먼저 getNode(pos)를 호출해야 함

```
def getNode(self, pos) :
    if pos < 0 : return None
    node = self.head;
    while pos > 0 and node != None :
        node = node.link
        pos -= 1
    return node
```

```
def getEntry(self, pos) :
    node = self.getNode(pos)
    if node == None : return None
    else : return node.data
```

삽입 연산: insert(pos, E)



① E를 이용해 새로운 노드 n을 만듦: n = Node(E, None)

② n의 다음 노드가 before의 다음 노드가 됨: n.link = before.link

③ before의 다음 노드가 n이 됨: before.link = n

```
def insert(self, pos, elem) :
    before = self.getNode(pos-1)
    if before == None :
        self.head = Node(elem, self.head)
    else :
        node = Node(elem, before.link)
        before.link = node
```

- 배열구조에서의 복잡도: O(n)
- 만약 before를 알고 있다면?
  - O(1)

삭제 연산: delete(pos)



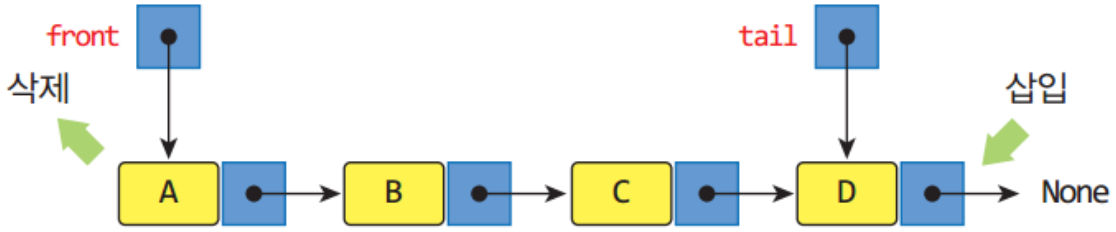
① before의 link가 삭제할 노드의 다음 노드를 가리키도록 함 : bofore.link = before.link.link

```
def delete(self, pos) :
    before = self.getNode(pos-1)
    if before == None :
        if self.head is not None :
            self.head = self.head.link
    elif before.link != None :
        before.link = before.link.link
```

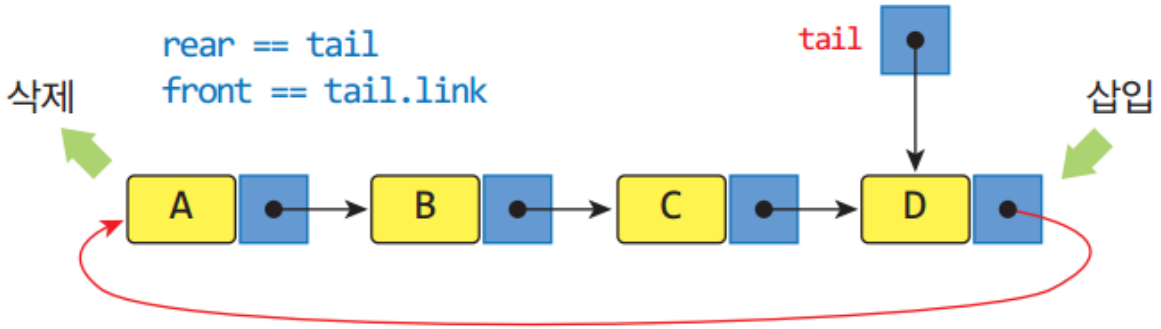
- 복잡도: O(n)
- 만약 before를 알고 있다면?
  - O(1)

6.4 원형연결리스트의 응용: 연결된 큐

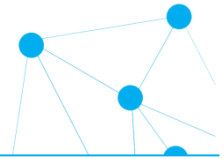
- 단순 연결 구조의 큐



- 원형 연결 구조의 큐
  - 머리 노드와 꼬리 노드를 모두 저장하고 있을 필요가 없음
  - tail만 저장해도 rear 와 front에 바로 접근할 수 있음.



LinkedQueue 클래스



- 클래스 정의와 생성자

```
class LinkedQueue:
    def __init__( self ):
        self.tail = None

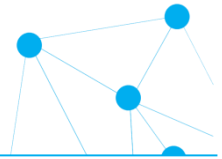
    def isEmpty( self ): return self.tail == None
    def isFull( self ): return False
```

연결된 큐의 생성자. 용량을 지정할 필요 없고, 데이터 멤버로 후단 노드를 가리키는 tail을 가짐

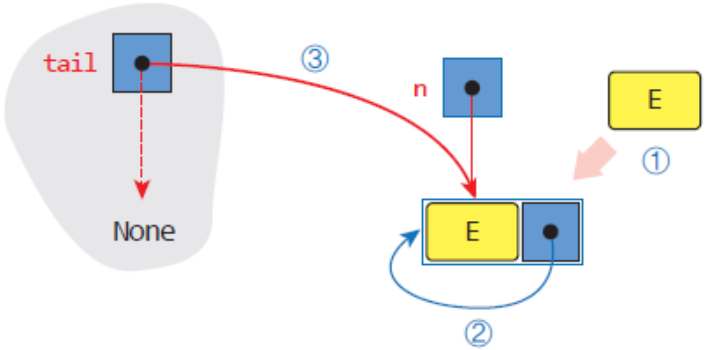
tail이 None이면 공백상태

연결된 구조에서 포화상태는 의미 없음

삽입 연산: enqueue()

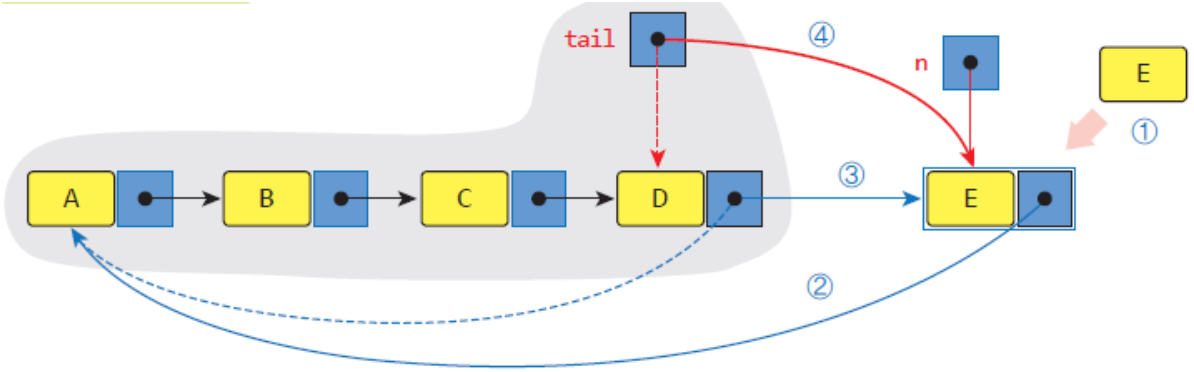


- Case1: 큐가 공백인 경우



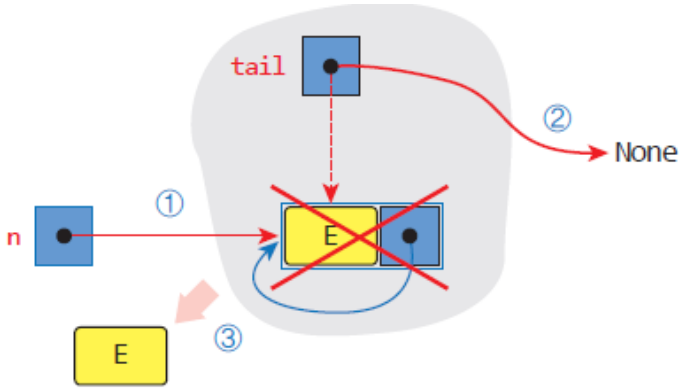

```
def enqueue( self, item ):
    node = Node(item, None)①
    if self.isEmpty() :
        self.tail = node ③
        node.link = node ②
    else :
        node.link = self.tail.link ②
        self.tail.link = node ③
        self.tail = node ④
```

- Case2: 공백이 아닌 경우



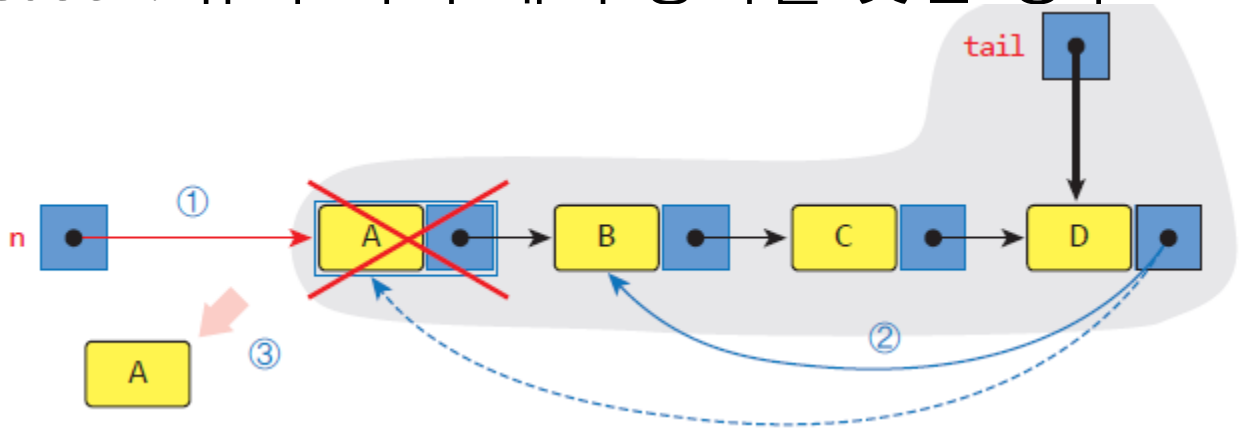
삭제 연산: dequeue()

- Case1: 큐가 하나의 항목을 갖는 경우



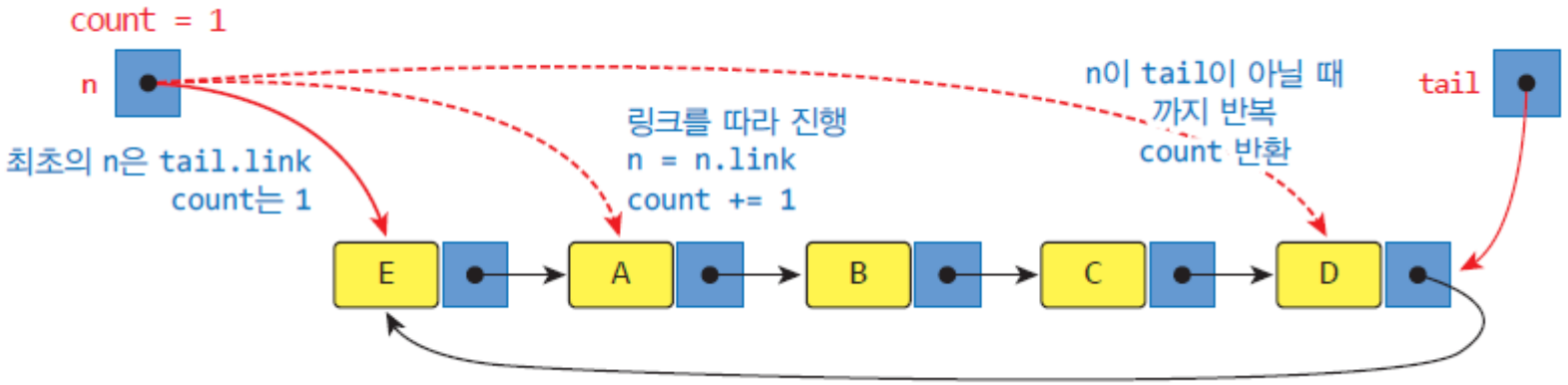

```
def dequeue( self ):
    if not self.isEmpty():
    ① data = self.tail.link.data   # 반환할 데이터
        if self.tail.link == self.tail :
            self.tail = None   ②
        else:
            self.tail.link = self.tail.link.link   ②
    ③ return data
```

- Case2: 큐가 여러 개의 항목을 갖는 경우



기타 연산들

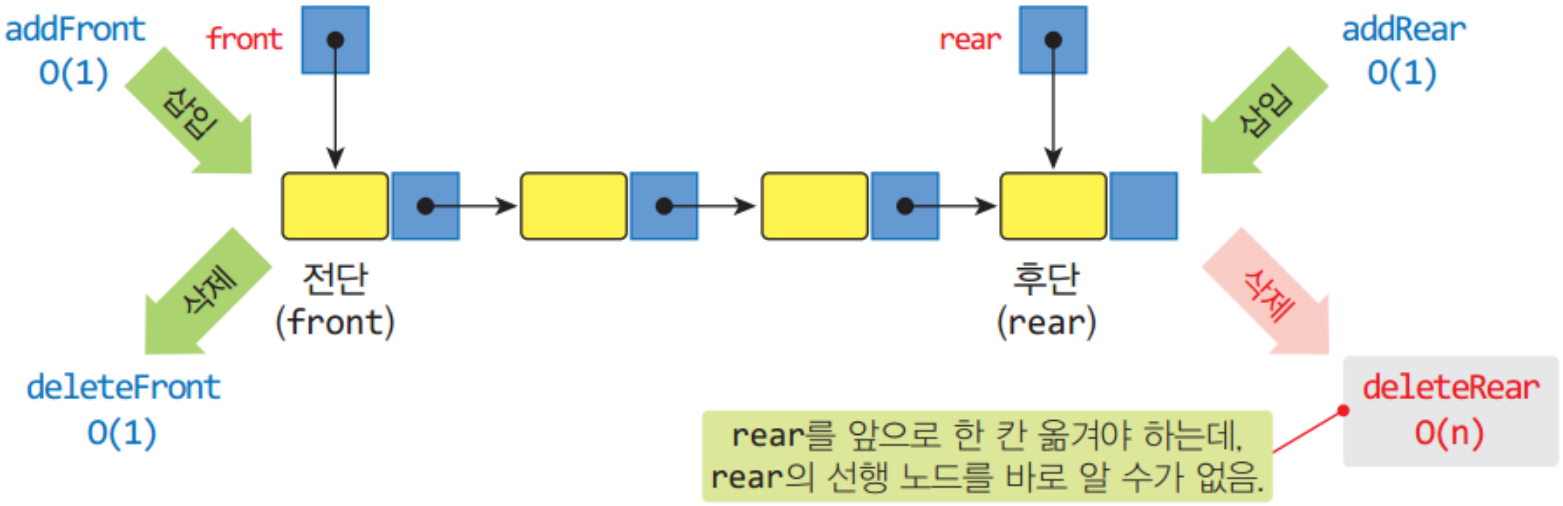
- 큐의 요소의 수 계산 size()
  - 종료 조건에 유의해야 함
  - 코드 6.7



- 문자열 변환을 위한 str 연산자 중복
  - 코드 6.8

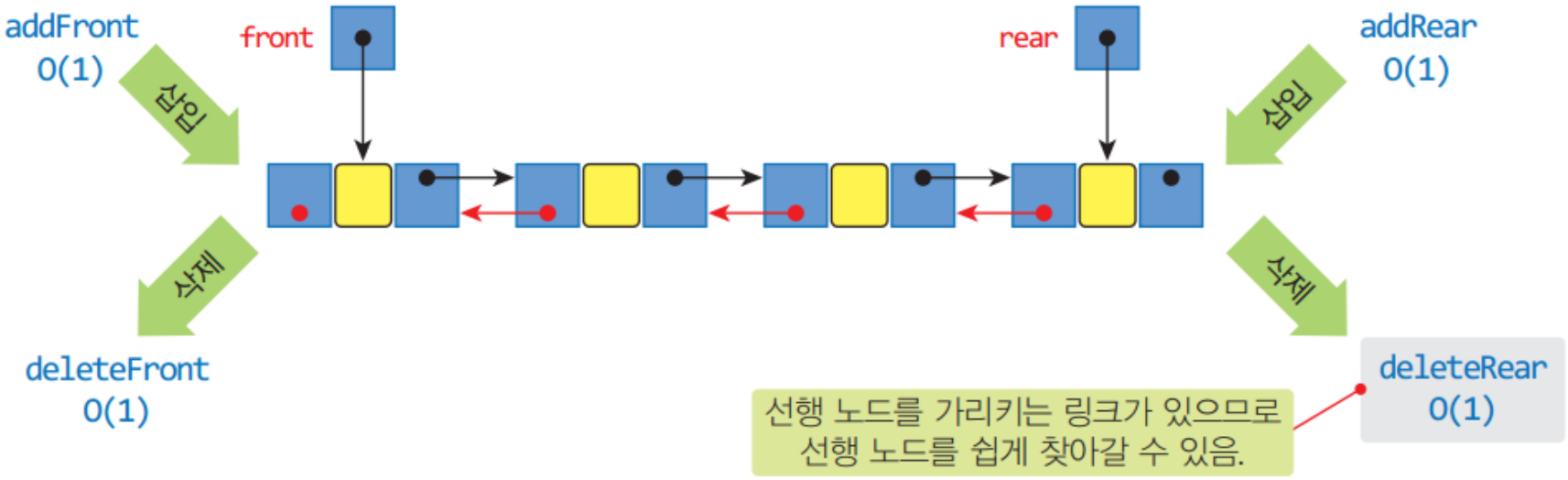
6.5 이중연결리스트의 응용: 연결된 덱

- 단순 연결 구조로 구현한 덱의 문제점?



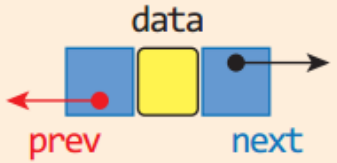
- 해결 방안은?
  - 이중연결리스트 사용

이중 연결 구조로 구현한 덱

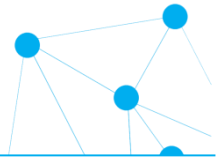


- 이중 연결 구조를 위한 노드 클래스

```
class DNode:                                    # 이중연결리스트를 위한 노드
    def __init__ (self, elem, prev = None, next = None):
        self.data = elem
        self.prev = prev
        self.next = next
```

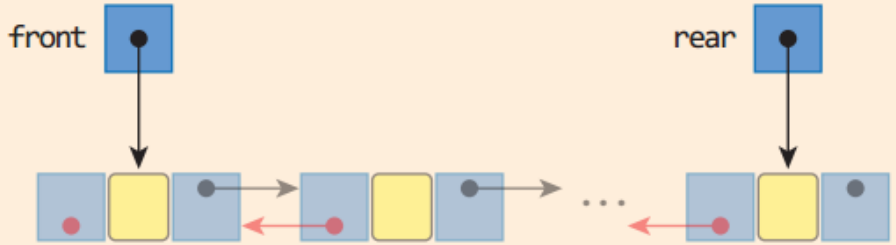


DoublyLinkedDeque 클래스



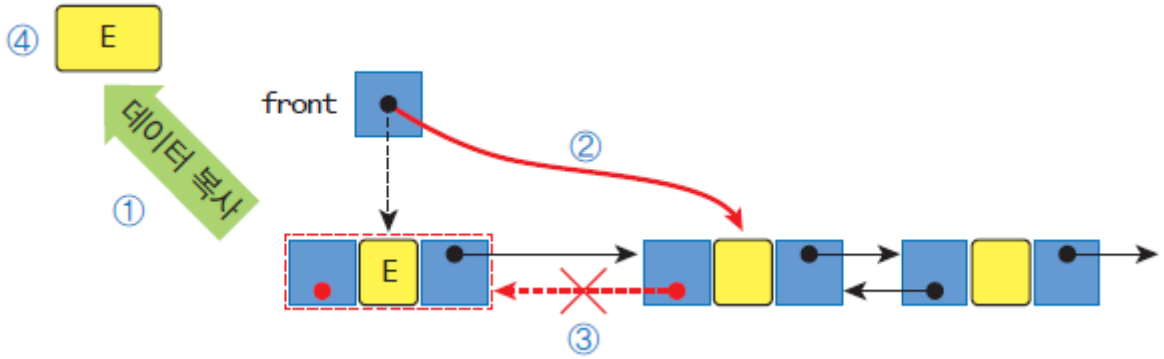
```
class DoublyLinkedDeque:
    def __init__( self ):
        self.front = None
        self.rear = None

    def isEmpty( self ): return self.front == None                    # 공백상태 검사
    def clear( self ): self.front = self.front = None                 # 초기화
    def size( self ) : ...                  # self.top->self.front로 수정. 코드 동일
    def display(self, msg):...              # self.top->self.front로 수정. 코드 동일
```



삽입 연산

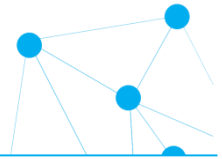
- 전단 삽입 addFront()



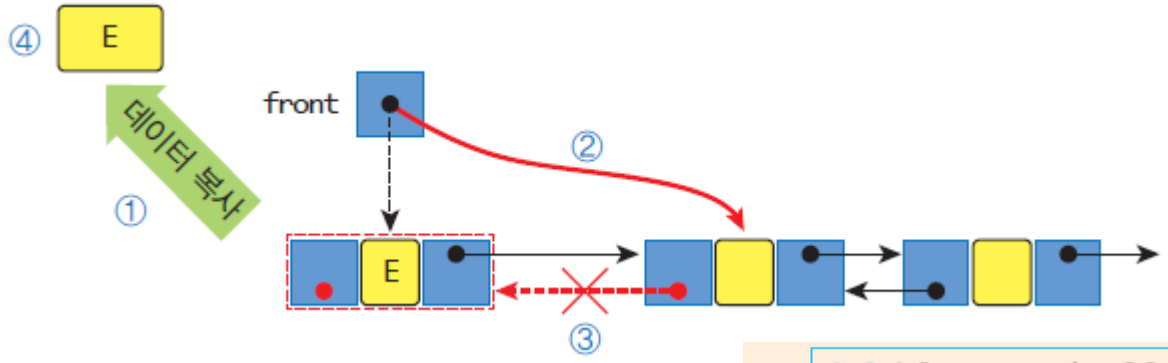

```
def addFront( self, item ):
    node = DNode(item, None, self.front)        # ①
    if( self.isEmpty()):
        self.front = self.rear = node
    else :
        self.front.prev = node                  # ②
        self.front = node                       # ③
```

- 후단 삽입 addRear()

삭제 연산



- 전단 삭제 deleteFront()




```
def deleteFront( self ):
    if not self.isEmpty():
        data = self.front.data              # ①
        self.front = self.front.next        # ②
        if self.front==None :
            self.rear = None
        else:
            self.front.prev = None          # ③
        return data                         # ④
```

- 후단 삭제 deleteRear()



감사합니다!